Reflective Journal: Lab 06 – Regression and Classification

Course: ITAI-1371

Authors: Arinze N Okafor, Hamza Ajmal,

Jonathan Wilturner

Lead Author: Arinze Okafor

Arinze Okafor

1. Conceptual Shift: Regression vs. Classification

Prior to this lab, supervised learning often blended together into a generic idea of "training a model on labeled inputs." Working through these two tasks highlighted how fundamentally the problem formulation dictates every downstream implementation choice.

With **Linear Regression**, the target (Fare) is an unbounded numerical continuum. The model is estimating a point on a continuous line. In contrast, **Logistic Regression** for passenger survival (Survived) is solving a boundary and probability problem. Seeing Logistic Regression in practice reinforced why the name can be deceptive—while it uses a linear combination of features internally, the output is passed through an S-shaped curve (sigmoid) to map predictions between 0 and 1 for discrete classes.

2. Workflow Complexity and Data Preparation

The pipeline required specific preprocessing adjustments for each algorithm:

- **Handling Missing Data:** Dropping null values from the Age column was essential. Neither model in scikit-learn handles NaN values out of the box, demonstrating that raw data must be cleaned before any mathematical optimization occurs.
- **Feature Encoding:** Linear Regression handled raw numeric columns (Age, Pclass) directly, but Logistic Regression required converting Sex from string labels into binary numerical values (0 and 1). This underscored that machine learning models only process numeric matrices, making feature preparation an active design decision rather than an afterthought.

3. Metric Alignment and Interpretation

The biggest operational insight came from analyzing model performance and weights:

- **Loss vs. Match:** Evaluating the regression model with Mean Squared Error (MSE = 3364.92, RMSE ≈ $58.01) made practical sense because regression error is about distance and degree of variance. Trying to evaluate fare predictions with accuracy would fail completely because predicting $32.50 when the true fare is $32.00 is functionally useful, but a strict accuracy test would count it as zero percent correct.
- **Accuracy Thresholds:** For the survival task, obtaining 74.83% accuracy provided an immediate baseline for how well the three features separated the two groups on the test set.
- **Model Coefficients (.coef_):** Examining the learned slopes confirmed expected domain trends—specifically, how a higher numerical passenger class (moving toward 2nd and 3rd class) exerted a negative weight on predicted ticket price.

4. Technical Takeaway

A working machine learning pipeline is only as reliable as the alignment between target type, preprocessing, and evaluation metrics. The workflow in this lab demonstrated that selecting an algorithm is straightforward; the real engineering lies in encoding features properly, splitting data cleanly without leakage, and picking loss metrics that truly reflect the physical meaning of the predictions.

Hamza Ajmal

Module 06 Lab – Reflective Journal

Regression and Classification

Understanding the two prediction tasks

This lab helped me distinguish regression from classification by focusing on the type of answer each model predicts. Linear regression estimates a continuous numerical value, which in this exercise is a passenger's Fare. Logistic regression predicts a class label, which here is Survived (0 or 1). The names initially seem confusing because logistic regression contains the word "regression," but its purpose in this lab is classification.

Training and evaluating linear regression

For the regression task, Age and Pclass are the input features and Fare is the target. The data is split into training and testing portions so the model is evaluated on rows it did not use for fitting. Mean squared error measures the average squared difference between predicted and actual fares. Because it squares the errors, larger mistakes receive a greater penalty. The square root of MSE gives an error measure in fare units, which is easier to interpret. The learned coefficients describe how the model's prediction changes with each feature when the other feature is held constant; they do not prove that the features cause fare differences.

Training and evaluating logistic regression

For classification, Sex is encoded as 0 for male and 1 for female, and Age, Pclass, and Sex are used to predict Survived. Logistic regression estimates class probabilities and assigns a class

based on its decision threshold. Accuracy is the proportion of test examples assigned the correct label. It makes sense here because the target is a category, whereas accuracy would not be useful for Fare prediction: a predicted fare can be close to the actual value without being exactly equal.

Workflow complexity and troubleshooting

The notebook also showed why the order of operations matters. The dataset must be loaded, the relevant columns prepared, the features and target selected, and the data split before fitting a model. A failure in an earlier cell can prevent later variables from being created, so running cells in order and reading the first error is more effective than trying to fix every later error separately. Stratifying the classification split helps keep the survival-class proportions similar in the training and test sets.

What I would consider next

The models in this lab are introductory baselines, not complete solutions to predicting fares or survival. Fare may depend on variables not included here, and the relationship between Age, Pclass, and Fare may not be purely linear. Accuracy also does not show which type of classification error is more common, so a confusion matrix, precision, recall, and F1-score could provide more detail. In a larger workflow, I would compare baseline models, inspect residuals for regression, and use a consistent preprocessing pipeline. I would also fit any learned preprocessing steps on training data only to avoid leakage.

Jonathon Wilturner

Reflective Journal: Module 6, Regression and Classification Models

Jonathon Wilturner | ITAI 1371

Introduction

In this lab I learned the difference between regression and classification, the two main kinds of supervised learning. We used the Titanic dataset to build a Linear Regression model that predicts what a passenger paid for a ticket and a Logistic Regression model that predicts whether a passenger survived. The lab showed me that the type of answer you want decides which model and which score you should use.

Description of the Experience

Our team worked through the lab in Google Colab using Python, pandas, and scikit-learn. We loaded the Titanic data, removed the rows with no age, and split it into 80 percent for training and 20 percent for testing.

For the regression model, we used Age and Pclass to predict Fare and scored it with Mean Squared Error. The result was 3364.92, and its square root, 58.01, told us the predictions were off by roughly $58 on a typical passenger.

For the classification model, we turned Sex into numbers (male as 0 and female as 1), used Age, class, and Sex to predict Survived, and scored it with accuracy. It came out at 74.83 percent. We then answered the three knowledge check questions in the notebook.

Personal Reflection and Learning

The main thing I took from this lab is that regression predicts a number and classification predicts a category. Fare is a number, so we measured how far off we were. Survival is a yes or no, so we counted how many predictions were right. Accuracy makes no sense for fares, because a prediction of $38.36 for a ticket that cost $13.00 is not right or wrong. It is just far or close.

The results also taught me to be careful with numbers. Our own output showed a predicted fare below zero for one passenger, which is impossible for a real ticket. A straight line does not know the real limits of the data. I also learned that 74.83 percent sounds good, but a score only means something when you compare it to something else.

Challenges, Improvements, and Questions

One challenge for me was knowing which score goes with which model. I worked it out by asking what kind of answer the model gives. If the answer is a number, like a fare, I use MSE to see how far off the guess is. If the answer is a category, like survived or did not survive, I use accuracy to count how many guesses were right.

Next time, I want to run the lab again with a different split of the data to see if the scores stay close. I also want to try adding more columns from the dataset and see if the scores get better.

One thing I still wonder is how 74.83 percent compares to a simple guess, like saying nobody survived. I want to learn how to test that, so I know how much the model really adds.

Conclusion

Overall, this lab showed me that building a model is only part of the job. I also have to prepare the data, pick the right score, and question the result before I trust it. As I build AI solutions for real clients through my own company, using the wrong measure could mean telling a client a model works when it does not. This lab gave me a clear habit to carry forward: match the model to the problem, match the score to the model, and always check the result.

Notes Before Submitting

Delete this page before you turn it in. It is for you only.

Where the numbers come from

- Every number in this journal comes from your team's Module 6 notebook: Mean Squared Error 3364.92, square root 58.01, accuracy 74.83%, the first predicted fare of 38.36 against an actual fare of 13.00, and a fifth predicted fare that starts with -0.3.
- I could not re-run the lab on the Titanic data, so these are team-reported, not rerun by me. What I could check: the square root of 3364.92 is 58.01, and 74.83% is about three out of four.
- To back it up yourself, rerun the cells in Colab and make sure your screen shows the same numbers. Colab sometimes changes results if a cell was run out of order.

What you must do

- Fill the yellow bracket in section 4 with a real challenge you ran into. If you had none, say that, and what you did to check your work instead.
- Sections 4 and 5 include a question and a plan. Keep them only if they are true for you. Change them to your own if you have different ones.
- The sentence in section 5 about your company is yours to keep or remove.

Before you submit

- Fill the yellow brackets with something true, then remove the yellow highlight.
- Read it out loud once. Change any sentence that does not sound like you or does not match what you did.

- Keep your own wording. Do not copy sentences from a teammate's journal.
- Export as PDF and name it to match your team's pattern, for example L06Journal_C_YourGroupName_ITAI1371.pdf or L06Journal_R_YourGroupName_ITAI1371.pdf. Put it in your team's GitHub repo and submit the repo link in Canvas.